Merge — Engineering Formation Platform

Vertical Slice Ticket List · 62 tickets across 9 categories

Version 1.1 · Updated post-contradiction resolution · June 2026 · Confidential

Priority guide: P1 = MVP core loop (must work before launch) P2 = Engagement (required for mindset shift) P3 = Platform quality (improves experience)

Category	Tickets	Priority	Key dependency
Identity & Onboarding	6	P1	None — build first
Scout Assessment	5	P1	ID-01
Curriculum & Content	9	P1	SC-04 (personalisation profile)
Assessment	15	P1	AI-01, CU-01, ID-03
Progression	5	P1	PR-01, AI-02, AI-04
Engagement	8	P2	ID-01
AI & Personalisation	8	P1	AI-01, INF-01, EN-01
Feedback — Clean Code	3	P3	AI-06 first, then AI-07, then AI-01
Async Infrastructure	3	P1	AI-01, INF-01

1. Identity & Onboarding

Authentication, student registration, GitHub OAuth, and encrypted token storage. Everything else depends on this category. Build it first.

ID	Title	Description	Priority
ID-01	**Supabase project setup**	Create Supabase project. Run the complete schema SQL. Enable pgvector extension. Configure Row Level Security policies on all student tables using auth.uid()::text = student_id::text. Verify RLS before any other work starts. No dependencies.	P1
ID-02	**Student registration endpoint**	POST /api/v1/auth/register. Accept name, email, phone, university email. Create student record with current_stage = SCOUT. Return JWT. Handle 409 on duplicate email, 400 on validation failure. Depends on ID-01.	P1
ID-03	**JWT auth and refresh**	POST /api/v1/auth/login (JWT on success, 401 on invalid). POST /api/v1/auth/refresh (silent token refresh, save work to localStorage if refresh fails). POST /api/v1/auth/logout (invalidate token). Protected route middleware on all authenticated endpoints. Rate limit: 5 auth attempts per minute per IP. Depends on ID-01.	P1
ID-04	**Student profile retrieval and update**	GET /api/v1/students/me — return full student record. PUT /api/v1/students/me — update profile fields. Both require valid JWT. Depends on ID-02, ID-03.	P1
ID-05	**GitHub OAuth flow and portfolio repo creation**	POST /api/v1/students/me/github/connect. Complete GitHub OAuth redirect and callback. Encrypt token with AES-256. Store in students.github_oauth_token_encrypted. Auto-create merge-portfolio public repo via GitHub API with auto_init: true. Store repo name on student record. Depends on ID-02.	P1
ID-06	**Gemini token encrypted storage**	POST /api/v1/students/me/gemini-token. Accept token from student. Validate it is a working Gemini API key before storing. AES-256 encrypt using key from environment variable only — never in database or frontend. Store in students.gemini_token_encrypted. Token must never be returned to the frontend after storage. Depends on ID-02.	P1

2. Scout Assessment

Three-layer intake assessment that produces the personalisation profile. No XP is earned here. The output of SC-04 drives every AI personalisation decision downstream.

ID	Title	Description	Priority
SC-01	**Layer 1 — background intake**	GET /scout/layer-1: serve 8 background intake questions in plain language. POST /scout/layer-1/submit: accept and persist responses to scout_assessments.layer1_responses (JSONB). No timer. Student answers in their own words. Depends on ID-02.	**P1**
SC-02	**Layer 2 — conceptual probe**	GET /scout/layer-2: serve 4 conceptual problems. No coding — pure thinking and reasoning. POST /scout/layer-2/submit: persist results to layer2_results. Captures thinking style and problem decomposition approach. Depends on SC-01.	**P1**
SC-03	**Layer 3 — technical baseline (conditional)**	GET /scout/layer-3: only shown if student indicated prior coding experience in Layer 1. Serve a small coding task to establish baseline. POST /scout/layer-3/submit: persist code to layer3_code. Skip gracefully if no prior experience — layer3_code remains null. Depends on SC-02.	**P1**
SC-04	**Personalisation profile generation**	Derive four attributes from Scout responses: thinkingStyle (SYSTEMATIC/INTUITIVE), motivationType (EXTERNAL/INTERNAL), priorExposure (NONE/SOME/EXPERIENCED), learningApproach (EXAMPLES_FIRST/DEFINITIONS_FIRST). Write PersonalisationProfile with initial scaffolding_level = 3, empty strength/weak concept arrays. This profile drives all AI calls downstream. Depends on SC-03.	**P1**
SC-05	**Stage placement**	POST /api/v1/scout/complete. Trigger SC-04 profile generation. On completion, set student.current_stage = CADET. Return profile summary and redirect target. This ticket closes the Scout stage permanently for the student. Depends on SC-04.	**P1**

3. Curriculum & Content

Stage and concept data, AI-generated written explanations, learning resources, and syntax exercises. CU-01 is a blocking bug — nothing else in this category works until it is fixed.

ID	Title	Description	Priority
CU-01	**Fix seed data bug — SCOUT to CADET**	BLOCKING BUG. All 12 Cadet concept rows in the seed INSERT statement use stage_name = 'SCOUT' instead of 'CADET'. Change every row to 'CADET'. Verify all 12 concepts load under the Cadet stage after the fix. Build this first — no other Curriculum ticket works until this is corrected. No dependencies.	P1
CU-02	**Stage retrieval endpoints**	GET /api/v1/stages — return all stages. GET /api/v1/stages/{stageType} — return stage details: xp_threshold, build_pass_score_threshold, ai_access_level, has_syntax_exercises, has_peer_review, clean_code_level. Depends on CU-01.	P1
CU-03	**Concept metadata endpoints**	GET /api/v1/stages/{stageType}/concepts — return ordered concept list with locked/unlocked state per authenticated student. GET /api/v1/concepts/{id} — return concept metadata: name, sequence_order, sfia_skill, failure_scenario, is_active. Return 403 if concept is not yet unlocked for this student. Depends on CU-01, ID-03.	P1
CU-04	**ConceptContent entity and data model**	Create the concept_content table: student_id, concept_id, explanation_text, generated_at, personalisation_version. Separate from the concepts table — this stores the AI-generated written explanation, which is unique per student. Add RLS policy. One record per student per concept, regenerated if personalisation profile version changes significantly. Depends on CU-01, SC-04.	P1
CU-05	**Concept content endpoint — AI-generated explanation**	GET /api/v1/concepts/{id}/content. Check if ConceptContent already exists for this student + concept. If yes, return it directly. If no, call CurriculumWriter prompt via AI-01 to generate a 500–800 word personalised written explanation, store in concept_content, return to client. Response includes: failureScenario (from concepts table), mergeExplanation (from ConceptContent). Depends on CU-04, AI-01.	P1
CU-06	**Learning resource delivery**	GET /api/v1/concepts/{id}/resources. Return all resources for the concept: title, type (video/audio/text/article), source, url, duration_minutes, is_recap, xp_value, offline_available, transcript_url. Resources are always supplementary — never required to pass a Drill. Depends on CU-03.	P1

ID	Title	Description	Priority
CU-07	**Learning resource completion and XP**	POST /api/v1/resources/{id}/complete. Check UNIQUE(student_id, resource_id) — award XP only once. Award 5 XP via Progression service. Enforce 50 XP cap per stage on learning_resource activity type. Return 200 { xpAwarded: 5 } first completion, 200 { xpAwarded: 0, alreadyCompleted: true } on repeat. Depends on CU-06, PR-01.	P1
CU-08	**Syntax exercise delivery — Cadet only**	GET /api/v1/concepts/{id}/syntax-exercise: return exercise only if student.current_stage == CADET, otherwise 404. Return broken_code, instructions, duration_minutes. POST /api/v1/syntax-exercises/{id}/submit: mark submitted, return correct/incorrect. No XP awarded — syntax exercises are not assessed. Not available from Engineer stage onwards. Depends on CU-03.	P1
CU-09	**Concept unlock logic**	After both Drills for a concept pass their comprehension checks, unlock the next concept in sequence. Update the concept unlock state so CU-03 returns the correct locked/unlocked status. Gate: both Drill 1 AND Drill 2 comprehension checks must pass, not just Judge0. Depends on AS-06.	P1

4. Assessment

The core learning loop. Drill retrieval, code reading gate, submission, Judge0 execution, comprehension check, and Build with five gates. AS-02 (code reading endpoint) is new — it was missing from the original spec.

ID	Title	Description	Priority
AS-01	**Drill retrieval with per-student generation check**	GET /api/v1/concepts/{id}/drills. Check if student-specific Drill 1 and Drill 2 records exist in drills table for this student + concept (drills table now has student_id). If yes, return them. If no, trigger AI-02 to generate and store them. Enforce Drill 2 locked until Drill 1 passes comprehension check. 403 if concept locked. Depends on CU-03, AI-02.	P1
AS-02	**Code reading submission endpoint**	POST /api/v1/drills/{drillId}/code-reading { response }. Required gate on Drill 2 only. Validates David has submitted a response to the code reading task before the code editor is accessible. Record the response. Return 200 { accepted: true } which	P1

ID	Title	Description	Priority
		signals the frontend to unlock the code editor. Without this endpoint returning 200, the code editor must not be shown. Depends on AS-01.	
AS-03	**Drill submission endpoint**	POST /api/v1/drills/{id}/submit { code, testSuite, architectureAnswers, idempotencyKey }. Validate: code present, testSuite non-empty, all architecture answers complete — reject with 400 before touching Judge0 if any fail. Check idempotency key — return 409 { originalSubmission } on duplicate. Verify student owns this drill and it is unlocked. Depends on AS-02 (for Drill 2 submissions), ID-03.	**P1**
AS-04	**Judge0 integration**	Send submitted code to Judge0 internal Docker network. language_id: 93 (JavaScript). cpu_time_limit: 2s. memory_limit: 262144 KB (256MB). enable_network: false — no exceptions. Poll GET /submissions/{token} for result. Map status codes: 3 = accepted (proceed to comprehension check), 4 = wrong answer (200, testsPassed: false), 5 = timeout (408), 6 = compilation error (400 with stderr to student), 11 = runtime error (400). Depends on AS-03.	**P1**
AS-05	**Comprehension check generation**	Trigger immediately when Judge0 returns status 3. Call AI-04 to generate questions from David's specific code — his actual variable names, function choices, architectural decisions. Questions must be unanswerable without understanding this specific implementation. Store in comprehension_checks with serverDeadline = triggered_at + (numQuestions × 10 seconds). Questions differ across attempts. Depends on AS-04, AI-04.	**P1**
AS-06	**Comprehension check submission and server-side timer**	POST /api/v1/comprehension-checks/{id}/submit { answers }. Compare submission timestamp against serverDeadline on the server — reject with 400 { timerExpired: true } if over. Client timer is visual only and never used for enforcement. AI scores answers against David's specific code. If passed: trigger XP award (PR-01) and async jobs. If failed: return 200 { passed: false }, student retries the Drill. Depends on AS-05, PR-01.	**P1**
AS-07	**Build unlock logic**	Check two conditions simultaneously: all Drills for the stage have passing comprehension checks AND student total_xp >= stage.xp_threshold. Both must be true. If conditions met: set build.is_unlocked = true, record unlocked_at, queue BUILD_PRD_GENERATION BullMQ job (AI-03). Build screen	**P1**

ID	Title	Description	Priority
		must show 202 until PRD generation job completes and prd is stored. Depends on AS-06, PR-01, INF-01.	
AS-08	**Build PRD delivery**	GET /api/v1/builds/current. Return the unlocked Build for the student's current stage including prd, requirements, constraints, sfia_competencies. 403 if Build not yet unlocked. 202 { generating: true } if unlocked but PRD job not yet complete. PRD is AI-generated and personalised to David's Drill performance history — not generic. Depends on AS-07.	**P1**
AS-09	**Build submission endpoint**	POST /api/v1/builds/{id}/submit { code, testSuite, architectureDocument, idempotencyKey }. Validate all three fields present and non-empty — reject 400 before processing. Check idempotency key — 409 on duplicate. Record attempt_number. Store build_submission. Trigger all five gates sequentially. XP decays by attempt: 1st 100%, 2nd 75%, 3rd 50%, 4th+ 25%. Depends on AS-08.	**P1**
AS-10	**Gate 1 — hidden test cases via Judge0**	Run submitted build code against hidden test cases via Judge0. Same Judge0 configuration as Drill execution. Record gate1_passed on build_submission. Do not reveal hidden test case content to student. Gate 1 must pass for any XP to be awarded. Depends on AS-09, AS-04.	**P1**
AS-11	**Gate 2 — student TDD test suite**	Execute the student's submitted test suite against their own code via Judge0. All submitted tests must pass. Cadet minimum: happy path + at least one edge case present. Verify test file is not empty and does not simply assert true. Record gate2_passed. Depends on AS-09.	**P1**
AS-12	**Gate 3 — Clean Code rubric score**	Call CleanCodeReviewer prompt via AI-01 to review submitted build code against the stage-appropriate rubric. Cadet: naming issues only. Engineer: +function size +redundancy. Architect: full SOLID principles. Score must meet the stage minimum threshold stored in stages.clean_code_level. Record gate3_passed and overall_score on build_submission. Depends on AS-09, AI-01.	**P1**
AS-13	**Gate 4 — Build comprehension check**	Same mechanism as Drill comprehension check. AI generates questions specifically from David's Build code — his design choices, his architecture document, his test strategies. 10-second server-side timer per question. serverDeadline	**P1**

ID	Title	Description	Priority
		enforced server-side. Record gate4_passed on build_submission. Depends on AS-09, AI-04.	
AS-14	**Gate 5 — SFIA alignment verification**	Verify build submission demonstrates competency in the SFIA skills declared in build.sfia_competencies. AI checks both the architecture document and the code against SFIA skill descriptors at the declared level. Pass requires evidence in both artefacts. Record gate5_passed. Depends on AS-09, AI-01.	**P1**
AS-15	**Build result aggregation and XP award**	Aggregate all five gate results. XP awarded by pass tier (after attempt decay): gates 1+2 only = 150 XP (minimum pass). Gates 1+2+3+4 = 300 XP (standard). All 5 gates = 500 XP (distinction). Award XP via PR-01. Queue GITHUB_COMMIT and COMPETENCY_SIGNAL BullMQ jobs. Return 200 { gates, xpAwarded, tier } immediately — async jobs run in background. Depends on AS-10 through AS-14, PR-01, INF-01.	**P1**

5. Progression

XP atomic transactions, cap enforcement, decay on retry, and two-gate stage promotion. Promotion checks XP threshold AND Build pass score simultaneously — nothing else. Graduation assessment does not gate promotion.

ID	Title	Description	Priority
PR-01	**XP award service — atomic transaction**	progressionService.awardXP(studentId, amount, source, stageType). BEGIN TRANSACTION. SELECT total_xp FROM students WHERE id = ? FOR UPDATE. Check current stage activity XP against cap. actualAward = MIN(amount × decayRate, remainingCap). UPDATE students.total_xp + INSERT xp_records (with sfia_skill, sfia_level, stage_type). COMMIT. Return { awarded, capped }. XP is never awarded from the frontend. Depends on ID-01.	**P1**
PR-02	**XP cap enforcement**	Read xp_caps table by stage_id and activity_type before awarding. Block any award when per-activity cap is reached. Return { awarded: 0, capped: true } on cap hit — never an error. Caps per Cadet stage: learning_resource 50 XP, drill_1 100 XP, drill_2 200 XP, weekly_consistency 150 XP, season_ranking 200 XP, build 500 XP. Depends on PR-01.	**P1**

ID	Title	Description	Priority
PR-03	**XP decay on retry**	Apply attempt decay before cap check. First attempt: 100%. Second: 75%. Third: 50%. Fourth and beyond: 25%. Track attempt_number from submissions and build_submissions tables. Pass decayRate to PR-01. This applies to both Drill and Build submissions. Depends on PR-01.	P1
PR-04	**Promotion eligibility check — two-gate only**	GET /api/v1/students/me/stage/promotion-status. Check both conditions simultaneously: student.total_xp >= stage.xp_threshold AND student Build pass score >= stage.build_pass_score_threshold. Return { eligible, missingXP, missingBuildScore } with exact numbers. Graduation assessment does not feature here — two-gate only. Depends on PR-01.	P1
PR-05	**Stage promotion execution**	POST /api/v1/students/me/stage/promote. Call PR-04 to verify eligibility. If both gates met: UPDATE student.current_stage to next stage, INSERT stage_promotions record (from_stage, to_stage, xp_at_promotion, build_score_at_promotion). Return 403 { missingXP, missingBuildScore } if not eligible. Return 409 if already promoted. Depends on PR-04.	P1

6. Engagement

Session management, mood-driven routing, Weekly Momentum Score, season management, and disengagement detection. These are P2 — required for the mindset shift mechanism but not for the core assessment loop.

ID	Title	Description	Priority
EN-01	**Session creation with mood**	POST /api/v1/sessions/start { mood: FRESH/OKAY/EXHAUSTED }. Validate mood against SessionMood enum — return 400 { validValues } on invalid. Create sessions record linked to student and current concept. Return { sessionId, conceptId, sessionType }. Depends on ID-03, CU-03.	P2
EN-02	**Session end**	POST /api/v1/sessions/{id}/end. Record ended_at, drills_completed, xp_earned for the session. Queue PERSONALISATION_UPDATE BullMQ job (AI-08) to process session patterns. Depends on EN-01, INF-01.	P2

ID	Title	Description	Priority
EN-03	**Mood-driven session routing**	On session start: FRESH → full session (failure scenario + explanation + resources + syntax exercise if Cadet + Drill 1 + Drill 2). OKAY → standard session (explanation + Drill 1 + Drill 2, skip syntax exercise). EXHAUSTED mid-concept → queue AUDIO_GENERATION job for 5–7 min reinforcement audio on current concept. EXHAUSTED at concept boundary → queue AUDIO_GENERATION for primer audio on next concept. Depends on EN-01, AI-05.	P2
EN-04	**Weekly Momentum Score calculation**	BullMQ MOMENTUM_CALCULATION job runs every Monday 00:00. For every student, calculate state from the past 7 days: DEPLOYING (3+ sessions, 90%+ Drill pass rate), BUILDING (3+ sessions, mixed pass rate), COMPILING (1–2 sessions any outcome), BLOCKED (0 sessions this week, had sessions last week), OFFLINE (0 sessions extended). Write to weekly_momentum. Reset on Monday. Depends on EN-01, INF-01.	P2
EN-05	**Cohort momentum visibility**	GET /api/v1/cohort/momentum. Return all students in the cohort with current momentum state, XP total, and initials (no full names). Momentum states are publicly visible to all cohort members. Used by the cohort strip on the dashboard. Depends on EN-04.	P2
EN-06	**Season management**	One season per university semester. Create season, set active, close at semester end. POST /api/v1/seasons (admin). On season close: lock all weekly_momentum records, freeze rankings. Closed seasons remain visible on student profiles permanently. Depends on ID-01.	P2
EN-07	**Season badge award**	SEASON_LOCK BullMQ job triggered at semester end. Calculate rank and percentile for all students in season by total_xp. Top 10%: GOLD badge + 200 XP bonus. Top 25%: SILVER badge + 100 XP bonus. INSERT season_badges. Award XP via PR-01. Badges are permanent — never removed from student profile. Depends on EN-06, PR-01, INF-01.	P2
EN-08	**Disengagement detection and reach-out**	BullMQ DISENGAGEMENT_CHECK job runs every 48 hours per student. If momentum state is BLOCKED or OFFLINE, call DisengagementCoach prompt via AI-01 to generate a personalised reach-out message referencing the student's actual journey and last active concept. Send via Intercom.	P2

ID	Title	Description	Priority
		Non-critical — skip gracefully on failure, retry once. Depends on EN-04, AI-01, INF-01.	

7. AI & Personalisation

All eight tickets are MVP scope — the AI personalisation engine is not deferred. AI-01 is the single Gemini call wrapper. AI-07 must be built before AI-01. Build AI-06 and AI-07 first, then AI-01, then everything that calls it.

ID	Title	Description	Priority
AI-01	**Gemini base service — single call wrapper**	The one place in the codebase that calls Gemini. Decrypt student's Gemini token via TokenEncryptionService. Call AI-07 to retrieve pgvector context before building any prompt. Build prompt from context + role-specific system instructions + student data. POST to Gemini API. Return response. Token never touches the frontend. Hard dependency for CU-05, AS-05, AS-12, AS-14, and all generation tickets. Depends on ID-06, AI-07.	**P1**
AI-02	**Drill generation**	Called by AS-01 when no student-specific drill exists. Build prompt from personalisation profile, weakness signals, hint patterns, coding style. Include a unique seed parameter per student per concept position — ensures no two students get identical drills. Call AI-01. Generate: PRD, architecture questions (3–5), code reading excerpt and task (Drill 2 only), MergeLabsCompany context. Store in drills table with student_id. Depends on AI-01, SC-04.	**P1**
AI-03	**Build PRD generation**	BullMQ BUILD_PRD_GENERATION job triggered when Build unlocks. Build prompt from the student's complete Drill performance history across the stage: weak concepts, comprehension scores by question type, hint usage patterns, Clean Code feedback patterns. Call AI-01. Output: personalised PRD that targets David's specific gaps. Store in builds.prd. Block Build screen with 202 until this job completes. Depends on AI-01, AS-06.	**P1**

ID	Title	Description	Priority
AI-04	**Comprehension question generation**	Called immediately when Judge0 returns status 3 (tests pass). Build prompt from David's specific submitted code — reference his actual variable names, function names, data structures, and architectural choices. Questions must be answerable only by someone who wrote or deeply understands this specific code. Call AI-01. Generate 4–6 questions. Questions differ across retry attempts. Depends on AI-01.	**P1**
AI-05	**Mood audio script generation**	BullMQ AUDIO_GENERATION job triggered on EXHAUSTED session. Build prompt from current concept, mood type (reinforcement vs primer), personalisation profile (learning approach, weak concepts). Call AI-01. Generate a 5–7 minute script. Store in audio_records. Deliver via Web Speech API on frontend — zero cost. Listening earns 5 XP. Recap audio type earns 0 XP. Depends on AI-01, EN-03, INF-01.	**P1**
AI-06	**pgvector embedding storage**	After each approved Drill submission and session end, generate a 1536-dimensional embedding from the interaction data (responses, code patterns, comprehension answers). UPDATE personalisation_profiles SET embedding = ?::vector WHERE student_id = ?. This embedding is what AI-07 queries for context retrieval. Must run before context retrieval can return meaningful results. Depends on AI-01, ID-01.	**P1**
AI-07	**pgvector context retrieval**	Called by AI-01 before every Gemini call. Query personalisation_profiles for the most similar embeddings: SELECT id, student_id, coding_style_patterns, hint_usage_pattern, weak_concepts FROM personalisation_profiles ORDER BY embedding <=> ?::vector LIMIT 5. Build context from results. This is what makes every Gemini response personalised to David's history. Build this before AI-01. Depends on AI-06.	**P1**
AI-08	**Personalisation profile update BullMQ job**	PERSONALISATION_UPDATE job triggered at session end by EN-02. AI analyses session patterns: hint usage count, comprehension scores by concept, pass/fail patterns, average time per drill, question types failed. Update personalisation_profiles: weak_concepts, strength_concepts, scaffolding_level, avg_session_duration, hint_usage_pattern, coding_style_patterns. Trigger AI-06 to generate updated embedding. Non-critical — skip on final failure, retry once. Depends on AI-01, AI-06, INF-01.	**P1**

8. Feedback — Clean Code

Async Clean Code feedback on every submission. P3 — does not block the core loop but is required for the full student experience. The feedback is phased by stage, specific and actionable, never a grade.

ID	Title	Description	Priority
FB-01	**Clean Code feedback generation — stage phased**	CleanCodeReviewer prompt via AI-01. Cadet: naming issues only, one specific issue per submission. Engineer: +function size +redundancy. Architect: full Clean Code rubric + SOLID principles. Principal: flag for human SeniorReview. Output: structured per-issue objects with issue type, affected line numbers, specific improvement suggestion. Never show a raw score prominently. Warm, specific, actionable. Depends on AI-01.	P3
FB-02	**Clean Code feedback delivery endpoint**	GET /api/v1/submissions/{id}/feedback. Return CleanCodeFeedback if ready (naming_issues, function_size_issues, redundancy_issues, solid_issues, overall_score). Return 202 { generating: true } if still generating. Student sees this on the Drill Result screen when ready — it does not block the submission 200 response. Depends on FB-01.	P3
FB-03	**Clean Code feedback BullMQ job**	CLEAN_CODE_FEEDBACK job queued asynchronously after comprehension check passes. Runs in background — does not block the 200 response David receives. 2 retries on Gemini failure. On final failure: query pgvector for similar previous feedback as fallback and use that. Depends on FB-01, INF-01.	P3

9. Async Infrastructure

BullMQ setup, GitHub commit job, and failure handling. INF-01 must exist before any async job ticket is built. These are prerequisites, not afterthoughts.

ID	Title	Description	Priority
INF-01	**BullMQ setup and queue definitions**	Redis connection configuration. Define all queue names: GITHUB_COMMIT, CLEAN_CODE_FEEDBACK, PERSONALISATION_UPDATE, COMPETENCY_SIGNAL, BUILD_PRD_GENERATION, AUDIO_GENERATION,	P1

ID	Title	Description	Priority
		MOMENTUM_CALCULATION, DISENGAGEMENT_CHECK, SEASON_LOCK, SPOT_CHECK_PEER_REVIEW. Worker configuration with concurrency settings per queue. This must exist before any ticket that queues an async job. Depends on ID-01 (Redis in docker-compose).	
INF-02	**GitHub commit job**	GITHUB_COMMIT BullMQ job. Decrypt GitHub token via TokenEncryptionService. PUT to GitHub API: /repos/{owner}/merge-portfolio/contents/{stage}/{concept}/drill-{type}.js. Commit message format: "Merge: {Stage} — {Concept} — {DrillType} — Passed". Store returned commit hash on the submission record. Retry 3× with 60s delay on 429 rate limit. Log to Sentry on final failure. Depends on INF-01, ID-05.	**P1**
INF-03	**Job failure handling and retry logic**	Dead letter queue for jobs exceeding retry limits. Sentry error logging on final failure for all jobs. Critical jobs (COMPETENCY_SIGNAL, BUILD_PRD_GENERATION) trigger an alert on failure — data must not be lost. Non-critical jobs (PERSONALISATION_UPDATE, AUDIO_GENERATION, DISENGAGEMENT_CHECK) log and skip on final failure. MOMENTUM_CALCULATION and SEASON_LOCK are scheduled — log errors only, no retry. Depends on INF-01.	**P1**

Open contradictions (do not close in code):

A2 (First Domino Problem) — remains open on the canvas. The handoff document's Berlin PR merge mechanism is not a confirmed answer. No engineering decision should be built assuming A2 is solved.